1) **JavaScript** is a high-level, dynamic, and programming language primarily used for building interactive and dynamic features on websites. It is one of the core technologies of web development, along with HTML and CSS, enabling the creation of responsive and engaging web applications. JavaScript is a versatile language that runs on the client-side (in the user's browser) and can also be used on the server-side (using Node.js).

Some key characteristics of JavaScript are:

Client-Side Scripting: JavaScript is commonly used to create interactive features in web pages, such as form validation, animations, dynamic content updates, and handling user events (like clicks or keyboard input).

Event-Driven Programming: JavaScript supports an event-driven model, where the code responds to user interactions or system events, making it ideal for building interactive applications.

Asynchronous Programming: With features like callbacks, promises, and async/await, JavaScript allows asynchronous operations, such as fetching data from a server without blocking the main thread, improving performance and user experience.

Cross-Platform: JavaScript runs on all major browsers (like Chrome, Firefox, Safari), making it a universal language for web development. It can also be used for building server-side applications with frameworks like Node.js.

Versatile: Beyond web development, JavaScript can be used for mobile app development (e.g., using React Native), desktop applications (e.g., using Electron), and even game development.

Dynamic Typing and First-Class Functions: JavaScript is dynamically typed, meaning you don't have to specify the type of a variable when declaring it. It also treats functions as first-class citizens, meaning functions can be assigned to variables, passed as arguments, and returned from other functions.

2) In JavaScript, **data types** are classified into **primitive and non-primitive** types. Primitive data types are basic, immutable types that hold a single value and include Number, String, Boolean, Null, Undefined, BigInt, and Symbol. These values are stored directly in the variable, and any operation on them creates a new value.

non-primitive data types, such as Object, Array, and Function, are mutable and can hold multiple values or complex structures. These are stored as references, meaning variables point to the same object in memory, so changes to one variable can affect others pointing to the same object.

3) JavaScript, **mutable** data types are those that can be changed or modified after creation, such as objects and arrays. When you alter a mutable value (e.g., adding items to an array or modifying an object's properties), it affects the original data structure directly.

Immutable data types, on the other hand, cannot be modified once they're created, such as primitive types like strings, numbers, and Booleans

4) In JavaScript, operators are symbols that perform operations on variables and values. They allow you to perform actions like calculations, comparisons, and logical evaluations.

5) In JavaScript, **Symbol()** is a unique and immutable data type used to create identifiers that are guaranteed to be distinct from any other. Symbols are often used as keys for object properties to avoid naming conflicts . symbols are not visible in loops or JSON serialization, providing a hidden, secure layer for data. They are especially useful in situations where you want to define private or special properties on an object without risking accidental access or overwriting by other parts of the code.

DOM

6) The **DOM (Document Object Model)** is a programming interface for web documents. It represents the structure of a webpage as a tree of objects, allowing developers to access and manipulate HTML and CSS elements dynamically using JavaScript. Each element, attribute, and piece of text in a document becomes a node in this tree. For instance, you can use JavaScript to update the content, style, or structure of a webpage without needing to reload it. Common DOM methods include `getElementById()`, `querySelector()`, and `createElement()`. These enable interaction with the webpage to create dynamic and interactive experiences

DOM Manipulation refers to the process of using JavaScript to modify, update, or interact with the elements of a webpage. This enables developers to create dynamic and interactive user experiences by changing the content, structure, and styles of the webpage in real-time.

Common Ways to Manipulate the DOM:

1. **Selecting Elements:**
   - getElementById(id) - Selects an element by its ID.
   - querySelector(selector) - Selects the first element matching a CSS selector.
   - getElementsByClassName(className) - Selects elements by class name.

```
const header = document.getElementById('header');

const items = document.querySelectorAll('.item');
```

2. **Changing Content:**
   - innerText or textContent - Modifies the text inside an element.
   - innerHTML - Changes the HTML inside an element.

-

```
header.innerText = 'Welcome to DOM Manipulation!';
```

3. **Adding or Removing Elements:**
    - createElement(tag) - Creates a new element.
    - appendChild(element) - Adds a child element.
    - removeChild(element) - Removes a child element.

```
const newDiv = document.createElement('div');
newDiv.innerText = 'Hello World!';
document.body.appendChild(newDiv);
```

4. **Modifying Attributes:**
    - setAttribute(name, value) - Sets an attribute.
    - getAttribute(name) - Retrieves an attribute.
    - removeAttribute(name) - Removes an attribute.

```
header.setAttribute('class', 'main-header');
```

5. **Changing Styles:**
    - Modify the style property directly.

```
header.style.color = 'blue';
header.style.fontSize = '24px';
```

6. **Adding or Removing Classes:**
    - classList.add(name) - Adds a class.
    - classList.remove(name) - Removes a class.

- classList.toggle(name) - Toggles a class.
-

```
header.classList.add('highlight');
```

7. **Event Listeners:**
   - Attach behaviors to elements when certain events occur.
   -

```
header.addEventListener('click', () => {
  alert('Header clicked!');
});
```

Event Listeners

Event Listeners in JavaScript are functions that listen for specific events, such as clicks, key presses, or mouse movements, on DOM elements. Using the addEventListener() method, you can attach a listener to an element to respond to an event. Once the event occurs, the callback function is triggered. You can also remove event listeners using removeEventListener(). Event listeners support both event bubbling (default behavior) and event capturing for controlling the propagation of events. This allows for flexible and efficient handling of user interactions or other events on a web page.

```
element.addEventListener('event-type', function, options);
```

- **Example:**

```
const button = document.querySelector('button');
button.addEventListener('click', () => {
 console.log('Button clicked!');
});
```

Event Propagation

Event propagation is the process by which an event flows through the DOM after being triggered. There are two main phases of event propagation:

1. **Capturing Phase:** The event starts from the outermost element and travels inward towards the target element.
2. **Bubbling Phase:** After the event reaches the target element, it then bubbles up from the target element to the root element.

By default, most events bubble from the target element back to the root, allowing parent elements to handle events triggered by child elements. However, event propagation can be controlled using methods like stopPropagation().

Stop Event Propagation

stopPropagation() is a method used to stop the event from propagating (bubbling or capturing) further through the DOM. When invoked on an event, this method prevents the event from triggering other event listeners that might be listening for the same event on ancestor elements. This is useful when you want to prevent certain actions or event listeners from firing while still handling the event on the target element.

```
element.addEventListener('click', function(event) {
  event.stopPropagation();  // Prevents the event from bubbling up
  console.log('Clicked!');
});
```

Stop Immediate Event Propagation

stopImmediatePropagation() is similar to stopPropagation(), but with a key difference. While stopPropagation() prevents the event from bubbling or capturing, stopImmediatePropagation() also stops any other event listeners attached to the same element from executing. This is especially useful when multiple event listeners are attached to the same element, and you want to ensure only one listener gets executed.

Example:

```
element.addEventListener('click', function(event) {
  event.stopImmediatePropagation();  // Stops other event listeners on the same element
  console.log('Clicked!');
});
```

---

40) Event Delegation

Event delegation is a technique in JavaScript where a single event listener is attached to a parent element instead of multiple listeners on individual child elements. The idea is to take advantage of event propagation (bubbling) to catch events on child elements from the parent. This approach is efficient because it reduces the number of event listeners in cases where many child elements need to be monitored for the same type of event.

```
const parent = document.querySelector('.parent');
parent.addEventListener('click', function(event) {
  if (event.target.matches('.child')) {
```

```
    console.log('Child element clicked!');
  }
});
```

In this example, the click event listener is attached to the parent element, and by checking if the target element matches a specific condition (the .child class), the event handler is executed for child elements. Event delegation is especially useful when dealing with dynamic content that gets added to the DOM after the initial page load.

Event Handling

Event handling refers to the process of responding to user interactions or other events in the browser, such as clicks, mouse movements, form submissions, etc. This is done by attaching event listeners to DOM elements, which will execute a function (callback) when the event is triggered. Event handling is a core concept in JavaScript, and it allows developers to create interactive and dynamic web pages.

There are two main ways to handle events in JavaScript:

1. **Using HTML attributes:** Directly in the HTML markup with event attributes like onclick, onmouseover, etc.
2. **Using JavaScript:** By attaching event listeners to elements using methods like addEventListener().

ARCHITECTURE

In JavaScript, when a program begins, it starts in what's called the **Global Execution Context**—a kind of **environment** where the code starts to execute. This context holds all global variables and functions and sets the initial reference for this, which in the browser context is typically window.

The execution context has two main components: the **variable environment** and the **thread of execution**. The **variable environment** (or memory phase) is where all variables and function declarations are stored in memory. This happens before any code is executed, allowing the interpreter to know which variables and functions are available. The **thread of execution** (or code phase) is the sequence in which the code is executed line by line, using the information stored in the variable environment.

JavaScript operates with two types of execution contexts: **global execution context** and **local execution context**. The global execution context is created by default when a JavaScript program starts and contains global variables and functions. Each function call creates its own local execution context, which is destroyed once the function execution is complete.

The execution context works in two phases:

1. **Memory Allocation Phase:** During this phase, JavaScript scans the code, allocates memory for variables and functions, and assigns undefined to variables until their actual values are assigned during execution. Functions, on the other hand, are fully stored in memory.
2. **Code Execution Phase:** In this phase, the JavaScript engine executes the code line by line, assigns values to variables, calls functions, and performs operations as specified.

As the program runs, functions are added to a structure called the **Call Stack**, which tracks which functions are currently running. Each time a function is called, a new execution context is created for that function and added to the call stack, which operates in a "last-in, first-out" manner. When a function finishes, it is removed from the stack, freeing it up for the next function to be executed.

For asynchronous tasks, JavaScript relies on additional browser features called **Web APIs** to handle operations like setTimeout, HTTP requests, and DOM events. When an asynchronous task completes, it doesn't go directly back into the call stack; instead, its callback function is sent to either the **Callback Queue** or the **Microtask Queue.**

The Callback Queue holds tasks like setTimeout or event listener callbacks, while the Microtask Queue is reserved for high-priority tasks, like those from promises.

This is where the **Event Loop** plays a crucial role. The event loop constantly monitors the call stack, waiting for it to clear. When the call stack is empty, the event loop first checks the microtask queue and pushes any available tasks to the call stack. If the microtask queue is empty, it then checks the callback queue for tasks to execute. This system allows JavaScript to handle asynchronous operations smoothly without blocking the main execution thread, making it appear as though multiple tasks are handled at once, even though JavaScript is **single-threaded**.

This process—moving between call stack, callback queues, and the event loop—enables JavaScript to efficiently manage both synchronous and asynchronous operations within a single-threaded environment

The **Event Loop** is a core part of JavaScript's runtime environment, enabling asynchronous operations to work seamlessly alongside synchronous code. It coordinates the execution of tasks, ensuring the program remains responsive and efficient.

The **Callback Queue** (or Task Queue) is where tasks like setTimeout, setInterval, and event listener callbacks wait to be executed. These tasks are added to the queue once their associated events or timers are completed. Similarly, the **Microtask Queue** handles higher-priority tasks, such as Promises or the Mutation Observer. Microtasks are processed before any task in the callback queue, ensuring a smooth flow for essential updates.

To prevent blocking, JavaScript uses a **Memory Heap** for dynamic memory allocation, managing objects and data.

(The **memory heap** is a region of a computer's memory where JavaScript stores objects, functions, and other dynamically allocated data during runtime. It is an essential part of the JavaScript engine's

memory management system, working alongside the call stack to handle the allocation and deallocation of memory.)

The **Just-In-Time (JIT) Compilation** optimizes performance by combining compilation and execution. Unlike traditional interpreters, which execute code line-by-line, or compilers, which process the entire code at once, JIT compiles code dynamically during execution for faster performance.

The **Abstract Syntax Tree (AST)** is an intermediate representation of code generated during parsing. It structures the code logically, enabling optimizations and transformations before execution.

JavaScript operates in a **Concurrency Mode**, which allows multiple tasks to proceed independently. While JavaScript itself is single-threaded, features like the event loop and Web APIs simulate concurrency, managing asynchronous tasks efficiently without blocking the main thread.

SCOPE

1) In JavaScript, scope refers to the accessibility of variables and functions within different parts of the code. Scope determines where a variable or function can be used and helps avoid conflicts in variable names by controlling their visibility.

Types of Scope

a) Global Scope: Variables declared outside any function or block are in the global scope, accessible from any part of the code. However, overusing global variables can lead to conflicts.

b) **Function Scope**: Variables declared within a function are only accessible inside that function, creating local variables isolated from the rest of the code.

c) **Block Scope**: Introduced with let and const in ES6, block scope restricts variable access to the block (such as within an if or for statement) where it was declared. This helps limit variable access to where it's needed, improving code clarity.

2) The **scope chain** is a mechanism JavaScript uses to resolve variable access. When a variable is accessed, JavaScript searches from the current scope outward, moving through function scopes up to the global scope until it finds the variable or reaches the global scope. This chain ensure that functions have access to variables in their own scope and in their outer, enclosing scopes, enabling nested functions to access outer variables while still respecting scope boundaries.

Lexical Scope in JavaScript refers to the scope in which a variable is defined based on its physical location in the source code. In lexical scoping, the scope of a variable is determined by where it is

declared, not where it is executed. This means that functions can access variables in their own scope as well as any outer (enclosing) scopes, forming a chain of scopes.

lexical environment is a data structure that stores all variables and function declarations within a specific scope. It helps the interpreter determine which variables and functions are accessible in different parts of your program.

A lexical environment consists of two key components:

Environment Record: An object that stores variables and functions declared within the current scope.

Reference to the Outer Environment: A pointer to the parent lexical environment, allowing access to variables and functions from outer scopes. d. Whenever a function or block is executed, a new lexical environment is created

3) In JavaScript, **shadowing** occurs when a variable in an inner scope has the same name as a variable in an outer scope, causing the inner variable to "shadow" or hide the outer one within that specific scope. This means that within the inner scope, the inner variable takes precedence, making the outer variable temporarily inaccessible. While shadowing can be useful for creating localized variables, it can lead to confusion if not managed carefully, especially in deeply nested scopes.

Shadowing in JavaScript occurs when a variable declared in a local scope (such as within a function or block) has the same name as a variable in an outer scope. The local variable "shadows" the outer one, meaning it takes precedence within its scope, while the outer variable remains accessible outside that scope. For instance, if a global variable x exists and a function declares another x locally, the local x is used within the function without affecting the global variable.

Illegal shadowing, however, happens when JavaScript's scoping rules are violated. This typically occurs when a let or const variable tries to shadow a var variable in the same scope, which is disallowed because var is function-scoped, whereas let and const are block-scoped. For example, declaring a let variable inside a block that attempts to shadow a var variable from the same scope is considered illegal shadowing. To avoid issues, it is recommended to use unique variable names and avoid mixing var with let or const in overlapping scopes.

8) **Hoisting** in JavaScript is a process where variable and function declarations are moved to the top of their scope before code execution. This allows functions and variables declared with var to be accessed before their declarations in the code, although var variables are initialized as undefined until assigned.

The **Temporal Dead Zone (TDZ)** refers to the period between entering a scope (like a block) and when a variable declared with let or const is initialized. During the TDZ, accessing these variables will cause a ReferenceError. This prevents usage of variables before they're declared, making the code safer and easier to understand.

FUNCTIONS

4) A **Function Statement** (also known as a function declaration) is a way to define a function in JavaScript using the function keyword, followed by the function name, parameters, and body. It is hoisted, meaning the function can be called before its declaration in the code.

A **Function Expression**, on the other hand, defines a function as part of an expression, often assigned to a variable. Function expressions can be anonymous (without a name) and are not hoisted, meaning they can only be called after they are defined in the code. The key difference is that function declarations are hoisted, while function expressions are not.

11) **Arrow Functions** and **Normal Functions** in JavaScript differ not only in syntax but also in behavior, particularly in terms of how they handle this, their capabilities, and where they are used.

a) Syntax: Arrow functions are concise and written using the => syntax. They allow developers to define functions in a shorter form, particularly useful for inline functions and callbacks

b) **Handling of this**: One of the most significant differences between arrow functions and normal functions is how they handle the this keyword:

Arrow Functions do not have their own this. Instead, they inherit this from their enclosing lexical scope. This means that inside an arrow function, the value of this is the same as it was in the outer function or object that defines the arrow function. This behavior is especially useful in situations like callbacks or event handlers where you want to retain the this context of the outer function.

Normal Functions, however, have their own this context, which depends on how they are called. This can lead to confusion, especially when used inside callbacks or event handlers

c) **Use Case and Flexibility**: Arrow functions are often used for shorter, simpler functions, especially where the context of this is important, such as inside methods, callbacks, or asynchronous operations. They are particularly useful for functional programming techniques and inline event handling. However, arrow functions cannot be used as constructors and don't support the new keyword

d) In contrast, **Normal Functions** are more versatile. They can be used as constructors (with the new keyword), allowing the creation of objects and providing a more dynamic approach to handling this in different contexts

e) **Hoisting**: **Normal functions** are hoisted, meaning the function declaration is moved to the top of its scope during the compile phase. This allows them to be called before they are defined in the code. Arrow functions, on t+
he other hand, are expressions and are not hoisted, so they can only be used after they are defined in the code.

12) In JavaScript, a **function** is not just a block of code that can be executed; it is also an **object**. This means that functions have properties and methods, just like other objects in JavaScript. Functions are instances of the Function constructor and can be assigned to variables, passed around as arguments, and even returned from other functions. They can have properties, such as name (the name of the function) and length (the number of parameters), and can also be called using methods like apply(), call(), and bind(). Since functions are objects, they can also have their own prototype, enabling the use of inheritance. This makes functions highly flexible and an essential part of JavaScript's functional programming capabilities. Functions as objects allow for dynamic behavior, such as adding custom properties to functions, creating closures, and building higher-order functions that manipulate other functions.

13) In JavaScript, a **generator function** is a special type of function that allows you to pause and resume execution, making it particularly useful for handling asynchronous tasks or creating iterators. Unlike regular functions, which run from start to finish once called, generator functions can yield multiple values one at a time, pausing between each yield and resuming when requested.

How to Define a Generator Function

A generator function is defined using the function* syntax, with an asterisk (*) after
the function keyword. Inside the function, you use the yield keyword to pause execution and return values one at a time.

How Generator Functions Work

A) Calling the Generator: When you call a generator function, it doesn't execute the code immediately. Instead, it returns an iterator called a "generator object," which can be used to control the function's execution.

A) Using the next() **Method**: The generator object has a next() method, which you call to begin or resume execution. When next() is called, the generator runs until it reaches
a yield statement. At that point, it pauses and returns an object with two properties:

- value: the yielded value.
- done: a Boolean indicating whether the generator has finished executing (true if finished, false otherwise).

14) In programming, a **pure function** is a function that consistently produces the same output given the same input and has no side effects. Pure functions are foundational in functional programming because they make code predictable, easy to test, and often more efficient

No Side Effects: The function does not modify any state outside of its scope, such as changing global variables, modifying input arguments, interacting with external APIs, or

15) An **anonymous function** is a function that is defined without a name. It is often used as a one-time callback or passed as an argument to other functions. Anonymous functions are commonly used in situations where the function does not need to be reused elsewhere in the code, making the code more concise and easier to manage.

16) A **First-Class Function** in JavaScript means that functions are treated like any other variable.

You can: • Store functions in variables.

• Pass them as arguments to other functions.

• Return them from functions.

17) A First Order Function in JavaScript is a regular function that doesn't take another function as one of its parameters and doesn't return another function within its body

In simpler terms:
• It accepts parameters (of different data types, either primitive or non-primitive).
• It may or may not return a value based on the calculations performed on the passed parameters.

18) In JavaScript, a higher-order function (HOF) is a function that can either:

Accept other functions as arguments:

*HOFs allow dynamic behavior by customizing their actions based on the function passed as an argument.

*Return a function as a result: HOFs create functions dynamically based on conditions or parameters. For instance:

19) **Self-invoking functions** in JavaScript are code snippets that execute immediately when called upon. They are also known as self-executing functions.

Here's a simple definition:

• A self-invoking function is defined as an anonymous function enclosed within parentheses, followed by another set of parentheses.

• It starts automatically without being explicitly called.

• You cannot self-invoke a function declaration; it must be a function expression.

20) A **Factory Function** in JavaScript is a function that returns a new object every time it is called. Unlike constructor functions, which use the new keyword to create instances of objects, a factory function simply returns an object directly. Factory functions are typically used to create multiple objects with similar properties and methods, avoiding the need to use new or this. They provide a flexible way to create objects, allowing you to define the properties dynamically and encapsulate the creation logic. One advantage of using a factory function is that it doesn't require the overhead of

creating a class or relying on inheritance. It also allows you to define private data and methods by using closures

21) In JavaScript, a **constructor** is a special function used to create and initialize objects. Constructor functions are typically defined with a capitalized name to distinguish them, and they are invoked with the new keyword. Inside the constructor, the this keyword refers to the new object being created, allowing properties and methods to be added to it.

Async vs Sync Functions in JavaScript

Synchronous Functions:
Synchronous functions execute in a sequential order, meaning each operation waits for the previous one to complete before moving on. If a function takes time (e.g., a long calculation or data fetch), it blocks the entire thread until it finishes. This behavior can lead to a frozen or unresponsive user interface in the browser.

Asynchronous Functions:
Asynchronous functions allow other tasks to run while waiting for a time-consuming operation to complete. They don't block the main thread, enabling better performance and responsiveness. Common examples include setTimeout, fetch, and Promise-based operation

21) A **callback function** in JavaScript is a function passed as an argument to another function, which is then executed after the completion of a certain task. Callback functions are commonly used in asynchronous programming to handle operations such as I/O tasks, user interactions, or fetching data from an API. For example, when reading a file or making an HTTP request, you might pass a callback to handle the result once the operation finishes.

However, **callback hell** (or "pyramid of doom") refers to a situation where multiple nested callbacks create complex, hard-to-read, and difficult-to-maintain code. Asynchronous operations often require callbacks to be nested within each other, leading to deep indentation and making the code look like a pyramid. This structure can become problematic, especially as the number of nested callbacks increases, leading to poor readability, bugs, and difficulty in debugging.

To mitigate callback hell, developers often use techniques like **promises** and **async/await**, which provide cleaner, more readable ways to handle asynchronous code and avoid excessive nesting.

PROMISE

22) A **Promise** in JavaScript is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It acts as a placeholder for a value that isn't available yet but will be resolved in the future. A Promise is in one of three states:

1. **Pending**: The Promise is still in progress, and the outcome (either success or failure) has not been determined yet.
2. **Resolved (Fulfilled)**: The Promise has completed successfully, and a value is available.

3. **Rejected**: The Promise has failed, and an error or reason for failure is provided.

`Promise.all` executes multiple promises concurrently and waits for all of them to resolve or reject. If any promise is rejected, the entire `Promise.all` is rejected.

Promise.allSettled waits for all promises to settle, regardless of whether they resolve or reject. It provides a detailed status for each promise.

Promise.race returns the result of the first promise to settle, whether it resolves or rejects.

Promise.any returns the first promise that fulfills (ignores rejections). If all promises are rejected, it throws an error.

23) A **Promise chain** is a way to perform a sequence of asynchronous tasks one after another, where each task depends on the result of the previous one. In a Promise chain, each .then() returns a new Promise, allowing us to perform the next asynchronous operation after the previous one has completed. This makes code more readable and easier to maintain compared to deeply nested callbacks.

Promises can be chained together using .then() and .catch() to handle multiple asynchronous operations sequentially or in parallel. Each .then() receives the result of the previous .then(), making it easy to process asynchronous tasks in a sequence without getting caught in callback hell.

24) **Async/await** is a more modern and cleaner way to work with Promises, introduced in ES2017. It allows you to write asynchronous code in a synchronous style. The async keyword is used to declare a function that will return a Promise, and await is used inside an async function to pause execution until a Promise is resolved or rejected. This simplifies asynchronous code by making it more readable and reducing the need for multiple .then() and .catch() chains

25) In JavaScript, the **this keyword** refers to the context or the object on which a function is invoked. Its value changes depending on how the function is called. In general, this refers to the object calling the function, but in different scenarios (such as within a function, in strict mode, or in arrow functions), it can point to different objects or contexts.

("The this keyword is commonly used in JavaScript to refer to the current context. In other words, this represents the object that is executing the current function.

For example, if you call a function inside an object, this will refer to that object. However, if you the same function in a global context (outside of any object), this will refer to the global object, which is window in a browser environment

23) JavaScript provides three powerful methods — **call**, **apply**, and **bind** — to explicitly set the value of this in functions:

(In JavaScript, call, apply, and bind are methods that help control the context of this inside functions.)

- **call**: This method calls a function with a specified this value and arguments provided individually. It's often used to borrow methods from one object for use in another.

```
function greet() {
  console.log(`Hello, ${this.name}`);
}
const person = { name: 'Alice' };
greet.call(person);  // Output: Hello, Alice
```

- **apply**: Similar to call, but it accepts arguments as an array rather than individually. This is useful for functions that accept an array of arguments.
-

```
function greet(greeting, punctuation) {
  console.log(`${greeting}, ${this.name}${punctuation}`);
}
const person = { name: 'Alice' };
greet.apply(person, ['Hello', '!']);  // Output: Hello, Alice!
```

- **bind**: This creates a new function with a permanently bound this value, allowing it to be invoked later with the defined context. It's useful for ensuring a specific this context in callbacks or event listeners.

```
const boundGreet = greet.bind(person);
boundGreet();  // Output: Hello, Alice
```

Using call, apply, and bind provides flexibility in managing this, making it easier to control context in different function calls.

24) **Currying** in JavaScript is a technique of transforming a function that takes multiple arguments into a series of functions, each taking a single argument. Instead of calling a function with all arguments at once, curried functions take one argument at a time and return a new function for the next argument, allowing partial application of arguments. For example:

```
function add(a) {
  return function(b) {
```

```
  return a + b;
 };
}
const addFive = add(5);
console.log(addFive(3));  // Output: 8
```

In this example, add is a curried function, enabling reuse and allowing arguments to be passed incrementally. Currying is useful in functional programming for creating reusable and customizable functions.

25) **Spread Operator (...)**

The spread operator is used to expand or "spread" elements of an iterable (like an array or object) into individual elements. It allows you to unpack elements or properties.

Use Cases of Spread:

1. Copying arrays:

   ```
   const arr1 = [1, 2, 3];
   const arr2 = [...arr1]; // Creates a shallow copy of arr1
   ```

2. Merging arrays:

   ```
   const arr1 = [1, 2];
   const arr2 = [3, 4];
   const mergedArr = [...arr1, ...arr2]; // [1, 2, 3, 4]
   ```

3. Spreading elements in function arguments:

   ```
   const numbers = [1, 2, 3];
   Math.max(...numbers); // Equivalent to Math.max(1, 2, 3)
   ```

4. Copying and merging objects:

   ```
   const obj1 = { a: 1, b: 2 };
   const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }
   ```

Rest Operator (...)

The rest operator collects multiple elements and packs them into an array. It’s typically used in function parameters to gather any number of arguments into an array or to capture the remaining elements of an array or object.

Use Cases of Rest:

1. Function parameters:

- The rest operator allows a function to accept any number of arguments as an array.

```
function sum(...numbers) {
   return numbers.reduce((acc, curr) => acc + curr, 0);
}

sum(1, 2, 3); // 6
```

2. Destructuring assignment:
   - In arrays, the rest operator gathers remaining elements.

```
const [first, ...rest] = [1, 2, 3, 4];
console.log(first); // 1
console.log(rest);  // [2, 3, 4]
```

   - In objects, it captures remaining properties.

```
const { a, ...others } = { a: 1, b: 2, c: 3 };
console.log(a);      // 1
console.log(others); // { b: 2, c: 3 }
```

Summary

- Spread (...): Expands elements of an array or object. Used for copying, merging, and spreading elements in function calls.
- Rest (...): Collects multiple elements into an array. Used in function parameters and destructuring to gather remaining elements or properties.

Rest Operator (... in Function Parameters): The rest operator is used in function definitions to gather multiple arguments into a single array parameter. This is useful when a function needs to accept a variable number of arguments without explicitly listing each one. The rest parameter can only be the last parameter in a function. It provides a way to handle flexible inputs while maintaining readability. For example:

```
function multiply(multiplier, ...numbers) {
  return numbers.map(num => num * multiplier);
}
console.log(multiply(2, 1, 2, 3));  // Output: [2, 4, 6]
```

Here, multiplier receives the first argument, while ...numbers gathers the rest into an array.

Spread Operator (... in Arrays and Objects): The spread operator expands or "spreads out" elements from an array or properties from an object. It's frequently used to copy arrays or objects, combine them, or pass individual elements as arguments to functions.

- **Array Example**:

```
const arr1 = [1, 2, 3];

const arr2 = [...arr1, 4, 5];  // Output: [1, 2, 3, 4, 5]
```

- **Object Example**:

```
const person = { name: 'Alice', age: 25 };

const newPerson = { ...person, location: 'Paris' };  // Adds location while copying other properties
```

Here, the spread operator copies elements from arr1 into arr2 and properties from person into newPerson. This technique is handy for making shallow copies of arrays or objects and combining properties without modifying the originals.

Rest Parameter: The **rest parameter** uses the rest operator in function declarations specifically to capture remaining arguments. Unlike the spread operator, which expands elements, the rest parameter "gathers" them into an array. It allows functions to take an unlimited number of parameters and manage them as a list, making it easier to process multiple inputs without creating rigid function definitions.

```
function greet(greeting, ...names) {

  return `${greeting}, ${names.join(' and ')}`;

}

console.log(greet('Hello', 'Alice', 'Bob', 'Charlie'));  // Output: "Hello, Alice and Bob and Charlie"
```

26) JavaScript, the concepts of **pass by value** and **pass by reference** define how arguments are passed to functions.

When a variable is passed by value, a copy of the value is sent to the function, meaning any changes made to the parameter inside the function do not affect the original variable outside it. This behavior applies to primitive data types such as numbers, strings, booleans, null, undefined, symbols, and bigints. For instance, if a number is passed to a function and modified within it, the original number remains unchanged outside the function because only its copy was affected.

On the other hand, when a variable is passed by reference, the function receives a reference or pointer to the actual memory location of the original object. This means any changes made to the object inside the function directly affect the original object. This behavior applies to non-primitive types such as objects, arrays, and functions. For example, if an object or an array is passed to a

function and modified, the changes will be reflected outside the function as both the parameter and the original variable point to the same memory address.

27) A **closure** in JavaScript is a feature that allows a function to remember and access its lexical scope, even when that function is executed outside of its original scope. This happens when an inner function keeps a reference to variables from its outer (or parent) function, effectively "closing over" them. Closures are created automatically whenever you define a function inside another function, and they are particularly useful for creating private variables and functions or for maintaining state.

28) An **object** in JavaScript is a complex data type that allows you to store collections of key-value pairs. Keys (or properties) in an object are strings or symbols, while the values can be any valid JavaScript data type, including numbers, strings, arrays, functions, or even other objects. Objects are one of the most fundamental building blocks of JavaScript, enabling the creation of reusable, organized, and dynamic data structures. Objects can be created using object literals ({}), the Object constructor, or through classes. Properties can be accessed and modified using dot notation (object.property) or bracket notation (object["property"]). JavaScript objects also support dynamic behavior, meaning properties can be added, updated, or deleted at runtime. Additionally, objects can have methods, which are functions assigned as values to properties, allowing encapsulation of behavior within the object. JavaScript also allows inheritance through prototypes, enabling object reuse and extending functionalities. In essence, objects form the backbone of most JavaScript programming, from basic data storage to defining more complex structures like classes and modules.

29) **Object methods** are functions that are defined inside an object and can be used to perform actions on or with the object’s data. They allow you to encapsulate behavior within an object, making it possible to manipulate and interact with the object's properties in an organized and reusable way.

In simple terms, methods are the "actions" that objects can perform. These methods can be predefined (built-in methods provided by JavaScript) or user-defined (custom methods created by developers).

1. **Object.keys(obj)**
   Returns an array of all the enumerable property keys of the object.

```
const obj = { a: 1, b: 2, c: 3 };

console.log(Object.keys(obj)); // Output: ['a', 'b', 'c']
```

2. **Object.values(obj)**
   Returns an array of all the enumerable property values of the object.

```
const obj = { a: 1, b: 2, c: 3 };

console.log(Object.values(obj)); // Output: [1, 2, 3]
```

3. **Object.entries(obj)**
   Returns an array of key-value pairs as nested arrays.

```
const obj = { a: 1, b: 2, c: 3 };

console.log(Object.entries(obj)); // Output: [['a', 1], ['b', 2], ['c', 3]]
```

4. **Object.fromEntries(entries)**
   Converts an array of key-value pairs back into an object.

```
const entries = [['a', 1], ['b', 2], ['c', 3]];

console.log(Object.fromEntries(entries)); // Output: { a: 1, b: 2, c: 3 }
```

5. **Object.assign(target, ...sources)**
   Copies properties from source objects to a target object and returns the updated target object.

```
const target = { a: 1 };

const source = { b: 2, c: 3 };

console.log(Object.assign(target, source)); // Output: { a: 1, b: 2, c: 3 }
```

6. **Object.freeze(obj)**
   Prevents modification of properties or adding new ones to the object.

```
const obj = { a: 1 };

Object.freeze(obj);

obj.a = 2; // Does nothing

console.log(obj); // Output: { a: 1 }
```

7. **Object.seal(obj)**
   Prevents adding or deleting properties but allows modification of existing ones.

```
const obj = { a: 1 };

Object.seal(obj);

obj.a = 2; // Allowed

delete obj.a; // Not allowed

console.log(obj); // Output: { a: 2 }
```

8. **Object.is(obj1, obj2)**
   Compares two values for equality, more robustly than ===.

```
console.log(Object.is(NaN, NaN)); // Output: true

console.log(Object.is(+0, -0)); // Output: false
```

9. **Object.hasOwn(obj, prop)**
   Checks if a property exists directly on the object (not inherited).

```
const obj = { a: 1 };

console.log(Object.hasOwn(obj, 'a')); // Output: true

console.log(Object.hasOwn(obj, 'b')); // Output: false
```

10. **Object.defineProperty(obj, prop, descriptor)**
    Adds or modifies a property on an object with detailed descriptors like writable, configurable, and enumerable.

```
const obj = {};

Object.defineProperty(obj, 'a', {

  value: 10,

  writable: false,

});

obj.a = 20; // Does nothing

console.log(obj.a); // Output: 10
```

11. **Object.defineProperties(obj, descriptors)**
    Defines multiple properties with descriptors at once.

```
const obj = {};

Object.defineProperties(obj, {

  a: { value: 10, writable: true },

  b: { value: 20, writable: false },

});

console.log(obj); // Output: { a: 10, b: 20 }
```

12. **Object.getOwnPropertyDescriptor(obj, prop)**
    Retrieves the descriptor for a specific property of the object.

```
const obj = { a: 1 };

console.log(Object.getOwnPropertyDescriptor(obj, 'a'));
```

```
/*
Output:
{
  value: 1,
  writable: true,
  enumerable: true,
  configurable: true
}
*/
```

13. **Object.getOwnPropertyDescriptors(obj)**
    Retrieves descriptors for all properties in the object.

```
const obj = { a: 1 };
console.log(Object.getOwnPropertyDescriptors(obj));
```

14. **Object.isFrozen(obj)**
    Checks if the object is frozen.

```
const obj = Object.freeze({ a: 1 });
console.log(Object.isFrozen(obj)); // Output: true
```

15. **Object.isSealed(obj)**
    Checks if the object is sealed.

```
const obj = Object.seal({ a: 1 });
console.log(Object.isSealed(obj)); // Output: true
```

16. **Object.create(proto, properties)**
    Creates a new object with the specified prototype and optional properties.

```
const parent = { a: 1 };
const child = Object.create(parent);
console.log(child.a); // Output: 1
```

30) Destructuring is a syntax in JavaScript that allows you to extract values from arrays or properties from objects and assign them to variables in a concise and readable way. It simplifies the process of accessing and using data, especially when working with complex data structures.

---

1. Array Destructuring

Array destructuring lets you unpack values from an array into individual variables based on their positions.

Example:

```
const numbers = [10, 20, 30];


// Destructuring assignment

const [a, b, c] = numbers;

console.log(a); // Output: 10

console.log(b); // Output: 20

console.log(c); // Output: 30
```

You can skip elements in the array by leaving blank spaces in the destructuring pattern:

```
const numbers = [10, 20, 30, 40];

const [first, , third] = numbers;

console.log(first); // Output: 10

console.log(third); // Output: 30
```

You can also use a rest parameter to collect the remaining elements:

```
const numbers = [10, 20, 30, 40];

const [first, ...rest] = numbers;

console.log(first); // Output: 10

console.log(rest);  // Output: [20, 30, 40]
```

---

2. Object Destructuring

Object destructuring lets you unpack properties from an object into variables using their property names.

Example:

```
const person = { name: "John", age: 25, country: "USA" };

// Destructuring assignment
const { name, age } = person;
console.log(name); // Output: John
console.log(age);  // Output: 25
```

You can assign properties to new variable names:

```
const person = { name: "John", age: 25 };
const { name: fullName, age: years } = person;
console.log(fullName); // Output: John
console.log(years);    // Output: 25
```

You can also provide default values for properties that don't exist:

```
const person = { name: "John" };
const { age = 30 } = person;
console.log(age); // Output: 30
```

---

3. Nested Destructuring

Destructuring works on nested objects and arrays as well.

Example with Objects:

```
const person = {
  name: "John",
  address: {
    city: "New York",
    zip: 10001,
  },
};
```

```
const {
  address: { city, zip },
} = person;
console.log(city); // Output: New York
console.log(zip);  // Output: 10001
```

Example with Arrays:

```
const numbers = [10, [20, 30], 40];
const [first, [second, third]] = numbers;
console.log(second); // Output: 20
console.log(third);  // Output: 30
```

---

4. Function Parameter Destructuring

You can use destructuring directly in function parameters to extract specific values.

Example:

```
const person = { name: "John", age: 25 };

function greet({ name, age }) {
  console.log(`Hello, my name is ${name} and I am ${age} years old.`);
}

greet(person); // Output: Hello, my name is John and I am 25 years old.
```

31) **Prototype chaining** in JavaScript is a mechanism that allows objects to inherit properties and methods from other objects through a chain of prototypes. Every object in JavaScript has a prototype, an object from which it can inherit methods and properties. If a property or method isn't found directly on an object, JavaScript will look for it on the object's prototype, and if not found there, it will keep searching up the chain until reaching null, which is the end of the chain. This chain of references is known as the prototype chain, enabling inheritance and shared behavior across instances.

In JavaScript, there are two main ways to access an object's prototype: __proto__ and prototype.

-

- **__proto__**: This is a reference to an object's prototype, which is typically used for accessing or navigating the prototype chain. Every object has an internal [[Prototype]] (often accessed as __proto__), which points to the object from which it inherits. This property is primarily used for lookup and inheritance purposes.
- **prototype**: The prototype property is a special property only found on functions (constructor functions in particular) and is used to define methods and properties that should be inherited by instances created with that function. When you create an object using a constructor function, the new object's __proto__ is set to the constructor's prototype, establishing a link in the prototype chain.
- **prototype inheritance** is a feature of JavaScript that allows objects to inherit properties and methods from other objects through a prototype chain. It is a core concept in JavaScript's object-oriented programming model, where each object can have a prototype—a reference to another object—which serves as a template from which it inherits properties and behaviors.

32) In JavaScript, a **constructor** is a special function used to create and initialize objects when called with the new keyword. Constructors are typically used in object-oriented programming to define blueprints for objects. By convention, constructor function names start with a capital letter to distinguish them from regular functions. When you call a constructor function with new, JavaScript creates a new object, assigns it to this, and links it to the function's prototype property so the new object can inherit properties and methods defined on that prototype. Finally, the constructor function initializes properties for the new object.

For example:

```
function Car(make, model) {
  this.make = make;
  this.model = model;
}

const myCar = new Car('Toyota', 'Corolla');
console.log(myCar.make);  // Output: "Toyota"
```

33) In JavaScript, **getters** and **setters** are special methods that allow you to define how properties of an object are accessed or modified. These methods provide a way to interact with object properties in a controlled manner, enabling data validation, transformations, or computations when values are read or written.

A **getter** is a method that allows you to retrieve the value of an object's property. It's defined using the get keyword, followed by the name of the property you want to access. It acts like a normal property when you access it, but it runs a function behind the scenes to return the value.

```
const person = {
  firstName: "John",
  lastName: "Doe",
  // Getter for fullName property
  get fullName() {
    return `${this.firstName} ${this.lastName}`;  } };
console.log(person.fullName); // Output: John Doe
```

A **setter** is a method that allows you to set or modify the value of an object's property. It’s defined using the set keyword and works by receiving a parameter representing the new value to be assigned to the property. You can add logic in the setter to validate or transform the value before assigning it to the property.

```
const person = {
  firstName: "John",
  lastName: "Doe",
  // Setter for fullName property
  set fullName(name) {
    const nameParts = name.split(" ");
    this.firstName = nameParts[0];
    this.lastName = nameParts[1];
  }
};
person.fullName = "Jane Smith";
console.log(person.firstName); // Output: Jane
console.log(person.lastName);  // Output: Smith
```

34) In JavaScript, **Map**, **WeakMap**, **Set**, and **WeakSet** are specialized data structures that provide efficient ways to store collections of data with unique characteristics:

- **Map**: A Map is a collection of key-value pairs where both keys and values can be of any type (including objects, arrays, and other maps). Unlike objects, maps maintain the insertion

order of entries and provide better performance for frequent additions and removals of key-value pairs. You can use methods like .set(), .get(), and .delete() to manage entries.

```
let map = new Map();

map.set('name', 'Alice');

console.log(map.get('name'));  // Output: "Alice"
```

- **WeakMap**: A WeakMap is similar to Map, but it only accepts objects as keys and doesn't prevent the garbage collection of those objects when they are no longer in use elsewhere. This makes WeakMap ideal for storing metadata or private data associated with objects, as it helps prevent memory leaks. WeakMap methods include .set(), .get(), and .delete(), but it cannot be iterated over or cleared.
- **Set**: A Set is a collection of unique values, meaning that duplicate values are automatically removed. Sets are useful for storing lists of items where duplicates are not allowed. You can add, remove, or check the existence of values using methods like .add(), .delete(), and .has(). Sets also preserve the insertion order of values.

```
let set = new Set();

set.add(1);

set.add(2);

set.add(2);  // Duplicates are ignored

console.log(set);  // Output: Set { 1, 2 }
```

- **WeakSet**: A WeakSet is similar to Set, but it only stores objects and does not prevent garbage collection of objects that are no longer referenced elsewhere. This makes WeakSet useful for temporary collections of objects, like tracking elements within a DOM or caching objects. Unlike Set, WeakSet is not iterable, nor does it support methods to list or clear entries.

35) In JavaScript, **objects** and **maps** are both used to store key-value pairs, but they have distinct characteristics and use cases. Objects are a fundamental part of JavaScript, primarily designed to represent structured data with named properties. Object keys are typically strings or symbols, and objects inherit properties and methods from their prototype, which can sometimes lead to unexpected behavior if keys overlap with inherited properties. Objects are generally suited for scenarios where you need a simple data structure with fixed key-value pairs, especially when the keys are known and fixed.

Maps, on the other hand, are a specialized collection designed explicitly for mapping unique keys to values. Maps can use any type of data (including objects, functions, and arrays) as keys, providing flexibility that objects do not. Additionally, maps maintain the insertion order of keys, which makes them more predictable for iterative tasks. Maps are also optimized for situations where keys are frequently added and removed, offering better performance in these cases. Unlike objects, maps do not inherit from a prototype, which avoids unintended property conflicts. Overall, maps are preferable when handling dynamic collections of data or when non-string keys are needed.

36) A shallow copy creates a duplicate of an object or array . copying only the top-level properties. It references nested objects rather than duplicating them . this means that changes to nested properties in the shallow copy will also affect the original object .

A deep copy creates a completely independent duplicate of an array or object including all nested objects and arrays . unlike a shallow copy which shares references for nested structures , a deep copy ensures that changes made to the copied object do not affect the original object.

37) JavaScript provides a rich set of methods to manipulate strings. These methods allow you to perform operations like searching, modifying, or transforming strings in various ways. Here's an overview of the most commonly used string methods:

---

1. .length

Returns the length (number of characters) of a string.

```
const str = "Hello, World!";

console.log(str.length); // Output: 13
```

---

2. .toUpperCase()

Converts all characters in the string to uppercase.

```
const str = "hello";

console.log(str.toUpperCase()); // Output: HELLO
```

---

3. .toLowerCase()

Converts all characters in the string to lowercase.

```
const str = "HELLO";

console.log(str.toLowerCase()); // Output: hello
```

---

4. .charAt()

Returns the character at a specified index in the string.

```
const str = "JavaScript";

console.log(str.charAt(4)); // Output: S
```

---

5. .indexOf()

Returns the index of the first occurrence of a specified value. If not found, it returns -1.

```
const str = "JavaScript";

console.log(str.indexOf("S")); // Output: 4

console.log(str.indexOf("z")); // Output: -1
```

---

6. .lastIndexOf()

Returns the index of the last occurrence of a specified value. If not found, it returns -1.

```
const str = "JavaScript is great!";

console.log(str.lastIndexOf("i")); // Output: 13
```

---

7. .slice()

Extracts a section of the string and returns it as a new string. It takes two arguments: the starting index (inclusive) and the ending index (exclusive).

```
const str = "JavaScript";

console.log(str.slice(0, 4)); // Output: Java
```

---

8. .substring()

Similar to slice(), but it does not accept negative indices. It returns a substring from the start index to the end index (exclusive).

```
const str = "JavaScript";

console.log(str.substring(0, 4)); // Output: Java
```

---

9. .replace()

Searches for a specified substring or pattern and replaces it with another string. It replaces only the first occurrence unless you use a global flag (/g) in the regular expression.

```
const str = "JavaScript is great!";

console.log(str.replace("great", "awesome")); // Output: JavaScript is awesome!
```

---

10. .replaceAll()

Replaces all occurrences of a specified substring or pattern with a new string.

```
const str = "JavaScript is great! JavaScript is awesome!";

console.log(str.replaceAll("JavaScript", "Python")); // Output: Python is great! Python is awesome!
```

---

11. .split()

Splits a string into an array of substrings based on a specified delimiter.

```
const str = "JavaScript, is, great";

console.log(str.split(", ")); // Output: ['JavaScript', 'is', 'great']
```

---

12. .trim()

Removes whitespace from both ends of a string.

```
const str = "  Hello, World!  ";

console.log(str.trim()); // Output: "Hello, World!"
```

---

13. .padStart()

Pads the current string from the left with another string until the resulting string reaches a specified length.

```
const str = "5";

console.log(str.padStart(3, "0")); // Output: "005"
```

---

14. .padEnd()

Pads the current string from the right with another string until the resulting string reaches a specified length.

```
const str = "5";

console.log(str.padEnd(3, "0")); // Output: "500"
```

---

15. .includes()

Checks if a specified value exists within the string. Returns true if found, otherwise false.

```
const str = "JavaScript is great";

console.log(str.includes("great")); // Output: true

console.log(str.includes("awesome")); // Output: false
```

---

16. .startsWith()

Checks if the string starts with a specified value. Returns true or false.

```
const str = "JavaScript is great!";
```

console.log(str.startsWith("Java")); // Output: true

---

17. .endsWith()

Checks if the string ends with a specified value. Returns true or false.

const str = "JavaScript is great!";

console.log(str.endsWith("great!")); // Output: true

---

18. .match()

Searches a string for a match against a regular expression and returns an array of results.

const str = "The quick brown fox";

console.log(str.match(/[a-z]+/g)); // Output: ["The", "quick", "brown", "fox"]

---

19. .repeat()

Returns a new string that repeats the original string a specified number of times.

const str = "Hello ";

console.log(str.repeat(3)); // Output: "Hello Hello Hello "

---

20. .normalize()

Returns a normalized version of the string, useful for handling different character encodings.

const str = "café";

console.log(str.normalize("NFC")); // Output: café

---

21. .fromCharCode() (Static Method)

Converts Unicode values into a string, where each Unicode value corresponds to a character.

console.log(String.fromCharCode(65, 66, 67)); // Output: ABC

---

22. .localeCompare()

Compares two strings in terms of their sort order in the current locale. It returns -1, 0, or 1 based on the comparison.

const str1 = "apple";

const str2 = "banana";

console.log(str1.localeCompare(str2)); // Output: -1 (apple comes before banana)

38)

Arrays are one of the most commonly used data structures in JavaScript. JavaScript provides a variety of built-in methods to manipulate arrays, making it easier to work with collections of data. Here's an overview of the most commonly used array methods:

---

1. .push()

Adds one or more elements to the end of an array and returns the new length of the array.

```
let arr = [1, 2, 3];

arr.push(4); // Adds 4 to the end

console.log(arr); // Output: [1, 2, 3, 4]
```

---

2. .pop()

Removes the last element from an array and returns that element.

```
let arr = [1, 2, 3];

const removedElement = arr.pop(); // Removes the last element

console.log(arr); // Output: [1, 2]

console.log(removedElement); // Output: 3
```

---

3. .shift()

Removes the first element from an array and returns that element.

```
let arr = [1, 2, 3];

const removedElement = arr.shift(); // Removes the first element

console.log(arr); // Output: [2, 3]

console.log(removedElement); // Output: 1
```

---

4. .unshift()

Adds one or more elements to the beginning of an array and returns the new length of the array.

```
let arr = [2, 3];

arr.unshift(1); // Adds 1 to the beginning
```

console.log(arr); // Output: [1, 2, 3]

---

5. .concat()

Joins two or more arrays and returns a new array without modifying the original arrays.

let arr1 = [1, 2];

let arr2 = [3, 4];

let combinedArr = arr1.concat(arr2);

console.log(combinedArr); // Output: [1, 2, 3, 4]

---

6. .join()

Joins all the elements of an array into a string and returns the string. You can specify a separator (default is comma).

let arr = [1, 2, 3];

console.log(arr.join('-')); // Output: "1-2-3"

---

7. .slice()

Returns a shallow copy of a portion of an array. It accepts two arguments: the starting index and the ending index (non-inclusive).

let arr = [1, 2, 3, 4, 5];

let newArr = arr.slice(1, 3); // Returns a new array from index 1 to 3

console.log(newArr); // Output: [2, 3]

---

8. .splice()

Changes the contents of an array by removing or replacing existing elements and/or adding new elements.

let arr = [1, 2, 3, 4, 5];

arr.splice(2, 2, "a", "b"); // Starts at index 2, removes 2 elements, adds "a" and "b"

console.log(arr); // Output: [1, 2, "a", "b", 5]

---

9. .forEach( )

Executes a provided function once for each element in the array.javascript

Copy code

```
let arr = [1, 2, 3];
arr.forEach((item) => {
  console.log(item); // Output: 1, 2, 3
});
```

---

10. .map()

Creates a new array populated with the results of calling a provided function on every element in the calling array.

```
let arr = [1, 2, 3];
let doubled = arr.map(item => item * 2);
console.log(doubled); // Output: [2, 4, 6]
```

---

11. .filter()

Creates a new array with all elements that pass the test implemented by the provided function.

```
let arr = [1, 2, 3, 4, 5];
let evenNumbers = arr.filter(item => item % 2 === 0);
console.log(evenNumbers); // Output: [2, 4]
```

---

12. .reduce()

Applies a function to each element in the array (from left to right) to reduce it to a single value.

```
let arr = [1, 2, 3, 4];
let sum = arr.reduce((accumulator, currentValue) => accumulator + currentValue, 0);
console.log(sum); // Output: 10
```

---

13. .reduceRight()

Same as .reduce(), but applies the function from right to left (starting with the last element).

```
let arr = [1, 2, 3, 4];
let sumRight = arr.reduceRight((accumulator, currentValue) => accumulator + currentValue, 0);
console.log(sumRight); // Output: 10
```

---

14. .some()

Tests whether at least one element in the array passes the test implemented by the provided function. Returns true if any element passes, otherwise false.

```
let arr = [1, 2, 3];

let hasEven = arr.some(item => item % 2 === 0);

console.log(hasEven); // Output: true
```

---

15. .every()

Tests whether all elements in the array pass the test implemented by the provided function. Returns true if all elements pass, otherwise false.

```
let arr = [2, 4, 6];

let allEven = arr.every(item => item % 2 === 0);

console.log(allEven); // Output: true
```

---

16. .find()

Returns the first element in the array that satisfies the provided testing function.

```
let arr = [1, 2, 3, 4];

let firstEven = arr.find(item => item % 2 === 0);

console.log(firstEven); // Output: 2
```

---

17. .findIndex()

Returns the index of the first element that satisfies the provided testing function. If no element satisfies the function, it returns -1.

```
let arr = [1, 2, 3, 4];

let index = arr.findIndex(item => item % 2 === 0);

console.log(index); // Output: 1
```

---

18. .includes()

Checks if an array contains a specific element, returning true if found, otherwise false.

```
let arr = [1, 2, 3];

console.log(arr.includes(2)); // Output: true

console.log(arr.includes(4)); // Output: false
```

---

19. .sort()

Sorts the elements of an array in place and returns the sorted array. By default, it sorts elements as strings.

```
let arr = [3, 1, 2];

arr.sort();

console.log(arr); // Output: [1, 2, 3]
```

---

20. .reverse()

Reverses the elements of an array in place.

```
let arr = [1, 2, 3];

arr.reverse();

console.log(arr); // Output: [3, 2, 1]
```

---

21. .copyWithin()

Shallow copies a portion of the array to another location in the same array.

```
let arr = [1, 2, 3, 4, 5];

arr.copyWithin(0, 3, 5); // Copies elements from index 3 to 5 to index 0

console.log(arr); // Output: [4, 5, 3, 4, 5]
```

---

22. .fill()

Fills all elements in an array with a static value, from a start index to an end index.

```
let arr = [1, 2, 3, 4];

arr.fill(0, 1, 3); // Fills elements from index 1 to 3 with 0

console.log(arr); // Output: [1, 0, 0, 4]
```

---

39)

Both for...of and for...in are loop constructs in JavaScript, but they are used for different purposes and have different behaviors. Here's a detailed comparison of both:

Key Differences between for...in and for...of:

Feature	for...in	for...of
Iterates over	Keys/indices (property names or array indices)	Values (values in iterable objects)
Works with	Objects, arrays, and enumerable properties	Arrays, strings, maps, sets, and other iterables
Ideal for	Accessing keys or indices, or enumerating object properties	Accessing values in an iterable (array, string, etc.)
Prototype Inheritance	Includes inherited properties from prototype chain	Does not include prototype properties
Order	Does not guarantee order (in case of arrays, the order is not always maintained)	Guarantees order (especially useful for arrays)

When to Use Each:

- Use for...in: When you need to loop over the keys of an object or array indices. It’s mostly used for objects, but can be used with arrays if you’re okay with working with the indices.
- Use for...of: When you need to loop over the values of an iterable (like an array or string). It's ideal for arrays, strings, and other iterable objects, and it provides a more readable and clear syntax.

Examples:

Using for...in with Object (Iterating Keys):

```
const car = { make: "Toyota", model: "Camry", year: 2020 };

for (let key in car) {

  console.log(key); // Outputs: make, model, year

}
```

Using for...of with Array (Iterating Values):

```
const numbers = [1, 2, 3, 4];

for (let num of numbers) {

  console.log(num); // Outputs: 1, 2, 3, 4

}
```

In conclusion, the choice between for...in and for...of depends on whether you need to work with keys/indices or the values of an iterable.

40)

41)

.

42) **setImmediate() and clearImmediate()**

setImmediate() is a method in JavaScript used to schedule a function to execute as soon as the current event loop completes, after any I/O events. It is part of the Node.js API and is not available in the browser environment. When you call setImmediate(), it queues the provided callback function to be executed immediately after the I/O events in the event loop, but before timers or other asynchronous operations.

- **setImmediate() Syntax:**

```
setImmediate(callback);
```

- **clearImmediate()** is used to cancel a previously scheduled immediate operation. If you call clearImmediate() with the ID returned by setImmediate(), the callback function will not be executed.
- **Example:**

```
const immediateId = setImmediate(() => {
  console.log('This will run immediately after the current event loop phase.');
});


clearImmediate(immediateId); // This will prevent the callback from executing
```

setInterval() and clearInterval()

setInterval() is a method that repeatedly calls a function or executes a block of code at specified intervals (in milliseconds). It runs the function continually until it is cleared using clearInterval(). This method is useful for tasks that need to occur periodically, such as updating the UI, refreshing data, or running a timer.

- **setInterval() Syntax:**

```
const intervalId = setInterval(callback, interval);
```

clearInterval() is used to stop a running interval. You pass the interval ID (returned by setInterval()) to clearInterval() to cancel the periodic execution.

- **Example:**

```
const intervalId = setInterval(() => {
  console.log('This runs every 1 second');
```

```
}, 1000);

// To stop the interval after 5 seconds
setTimeout(() => {
  clearInterval(intervalId);
  console.log('Interval cleared');
}, 5000);
```

---

setTimeout() and clearTimeout()

setTimeout() is a method that allows you to schedule a function to execute after a specified delay (in milliseconds). Unlike setInterval(), which repeats indefinitely, setTimeout() runs the specified function only once after the delay. It's commonly used for time-based actions, such as delaying the execution of a function or creating animations that start after a certain time.

- **setTimeout() Syntax:**

```
const timeoutId = setTimeout(callback, delay);
```

- **clearTimeout()** is used to cancel a scheduled timeout before it executes. By passing the timeout ID (returned by setTimeout()), the callback function is prevented from executing.
- **Example:**

```
const timeoutId = setTimeout(() => {
  console.log('This will run after 3 seconds');
}, 3000);

// To cancel the timeout before it executes
clearTimeout(timeoutId);
```

---

setTimeout() vs setInterval()

While both setTimeout() and setInterval() schedule code to run after a delay, they differ in behavior:

- **setTimeout()** executes the function **once** after the specified delay.
- **setInterval()** executes the function **repeatedly** at regular intervals, until it is stopped using clearInterval().

43) try...catch

The try...catch block is used to test a block of code for errors and to catch those errors when they occur. The code inside the try block is executed, and if an error occurs, it jumps to the catch block where the error can be handled.

- Syntax:

```
try {
  // Code that may throw an error
} catch (error) {
  // Code to handle the error
}
```

Example:

```
try {
  let result = someFunction();  // This function may throw an error
} catch (error) {
  console.log('An error occurred: ' + error.message);
}
```

If someFunction() throws an error, the program will not crash. Instead, the error message will be captured by the catch block, allowing the application to continue running.

---

throw

The throw statement is used to manually create custom errors or exceptions. It allows you to throw an exception and pass it to the catch block to be handled. This can be useful when you want to validate inputs, detect unusual conditions, or force an error under specific circumstances.

- Syntax:

```
throw new Error('This is a custom error message');
```

Example:

```
function checkAge(age) {
  if (age < 18) {
    throw new Error('You must be 18 or older');
  } else {
    console.log('Access granted');
  }
}
```

```
try {
  checkAge(16);  // This will throw an error
} catch (error) {
  console.log(error.message);  // Catch and handle the custom error
}
```

In this example, the throw statement is used to raise an exception if the provided age is less than 18, which is then caught and handled in the catch block.

---

finally

The finally block is an optional block that follows the try...catch structure. It executes code after the try and catch blocks, regardless of whether an error was thrown or not. The finally block is typically used to execute cleanup code, such as closing resources, freeing memory, or restoring application state.

- Syntax:

```
try {
  // Code that may throw an error
} catch (error) {
  // Code to handle the error
} finally {
  // Code that will run regardless of whether an error occurred
}
```

Example:

```
try {
  let result = someFunction();  // This function may throw an error
} catch (error) {
  console.log('An error occurred: ' + error.message);
} finally {
  console.log('This will always run, even if there is an error');
}
```

In this example, the finally block will always run, whether an error occurs in the try block or not, making it ideal for tasks like closing files or cleaning up resources.

44) **JS Engine Working**

A **JavaScript engine** is a program that reads and executes JavaScript code. It processes the code by parsing it into an Abstract Syntax Tree (AST), and then executing it. Some engines, like V8, use Just-In-Time (JIT) compilation to convert JavaScript into machine code for faster execution. The engine also handles garbage collection to free up memory when objects are no longer needed. JavaScript engines run in a **single-threaded** environment, meaning they process one operation at a time, but asynchronous functions allow non-blocking operations.

V8 Engine

The **V8 engine** is Google's JavaScript engine used in Chrome and Node.js. It compiles JavaScript directly into machine code using **JIT compilation**, making execution faster. V8 also manages memory with **garbage collection** and optimizes frequently used code to improve performance. It supports modern ECMAScript features and is known for its speed, which is a key reason for the fast performance of Chrome and Node.js.

Single vs Multi-Threading

Single-threading means only one task can run at a time. JavaScript is **single-threaded**, meaning it processes code in sequence, one after another. Asynchronous functions, like setTimeout() and Promises, allow JavaScript to handle other tasks without waiting for the current one to finish.

Multi-threading allows multiple tasks to run simultaneously. While JavaScript is single-threaded, **Web Workers** and **Worker Threads** allow running tasks in the background without blocking the main thread.

Starvation

Starvation occurs in multi-threaded environments when a thread is repeatedly denied access to resources, causing it to never get a chance to execute. This can happen if other threads are given priority continuously. In JavaScript, although it uses a single-threaded model, starvation can occur when using **Web Workers** or **Node.js worker threads** if they aren’t managed properly.

45) The **memory heap** is a part of memory in JavaScript where objects, arrays, and other dynamically allocated data are stored. Unlike the stack, which is organized and managed automatically with simple, short-lived data (like function calls and primitive values), the heap is used for more complex data structures and long-lived items.

46) The **Fetch API** in JavaScript is a modern way to make network requests, like retrieving data from a server, directly from the browser. It’s commonly used to send HTTP requests and handle responses in web applications

Both **Axios** and **Fetch** are used for making HTTP requests in JavaScript, but they differ in terms of functionality, ease of use, and features. Let's break down the key differences between Axios and Fetch

Fetch is a built-in browser API and does not require any external libraries. It uses Promises, making it a good choice for simpler use cases.

Axios is a third-party library that you need to install (via npm or CDN). It also returns Promises but offers more functionality out of the box, such as automatic JSON parsing and better error handling.

47) **Local Storage** allows you to store data persistently on a user's browser without an expiration time. Data stored in local storage remains available even after the browser is closed and reopened, making it ideal for saving user preferences or application settings that should be available across sessions. However, local storage is limited in size (usually around 5MB) and is only accessible to the domain that stored the data.

Session Storage is similar to local storage but with one key difference: data stored in session storage is only available for the duration of the page session. This means the data is cleared when the browser or tab is closed. It is useful for temporary data that should not persist beyond the current session, like storing form data while a …

48) **JSON** (JavaScript Object Notation) is a lightweight, text-based format used for storing and exchanging data between a server and a client. It is easy for humans to read and write and easy for machines to parse and generate, making it the most commonly used data format for APIs and web services.

JSON represents data in a key-value pair format, similar to JavaScript objects, but it's language-independent, meaning it can be used across different programming languages. JSON is primarily used to send and receive data in web applications.

The **JSON.stringify()** method converts a **JavaScript object** into a **JSON string**. This is useful when you need to send data from a JavaScript environment to a server, or save it to storage in a string format. It converts objects, arrays, and other values into a JSON format.

The **JSON.parse()** method does the opposite: it converts a **JSON string** back into a **JavaScript object**. This is useful when you retrieve data as a JSON string and want to work with it as an object in your code.

48) ES6 (ECMAScript 2015) introduced several powerful features :

A Let and Const: ES6 introduced let and const for block-scoped variable declarations, providing more control over variable scope compared to the traditional var.

B Arrow Functions: Arrow functions provide a shorter syntax for writing functions and preserve the this context from their surrounding scope, making them useful in callbacks and event handlers.

C Template Literals: Template literals allow for multi-line strings and variable interpolation using ${} syntax, making string manipulation more intuitive.

D Destructuring Assignment: This feature allows you to unpack values from arrays or properties from objects into distinct variables, making code cleaner and more readable.

E Default Parameters: ES6 enables you to set default values for function parameters, making it easier to handle missing arguments.

F Rest and Spread Operators: The rest operator (...) allows you to collect all remaining function arguments into an array, while the spread operator (...) makes it easy to spread elements from arrays or objects into new ones.

G Classes: ES6 introduced a more structured and object-oriented way to create classes and inheritance, using the class syntax.

H Modules: ES6 modules allow for cleaner and more maintainable code by supporting import and export to share functionality across files.

I Promises: Promises provide a cleaner and more manageable way to handle asynchronous operations compared to traditional callback functions, improving error handling and chaining.

49) The **Optional Chaining Operator** (?.) in JavaScript is a feature introduced in ES2020 that allows you to safely access deeply nested properties of an object without having to check if each reference in the chain is valid (i.e., not null or undefined). It provides a concise way to avoid errors when attempting to access properties or call methods on null or undefined values.

When you use optional chaining, if a reference in the chain is null or undefined, the expression short-circuits and returns undefined instead of throwing an error. This is particularly useful for working with data from APIs or objects with optional properties, where not all values are guaranteed to be present.

```
if (user && user.address && user.address.city) {

  console.log(user.address.city);

}
```

You can use optional chaining:

```
console.log(user?.address?.city);
```

In this case, if user, user.address, or user.address.city is null or undefined, it will safely return undefined without causing an error, making the code cleaner and more readable. The optional chaining operator can also be used with function calls and array indexing.

50)**Memoization** is an optimization technique used in programming to improve the performance of functions by caching the results of expensive function calls and reusing those results when the same inputs occur again. Instead of recalculating the result for the same input multiple times, memoization stores the computed result in a cache (usually an object or a map) and retrieves it quickly if the

function is called with the same arguments in the future. This is particularly useful for functions with repetitive or overlapping calculations, such as those involved in recursive algorithms or those that require intensive computation, like in dynamic programming problems.

51) Memory management in JavaScript refers to the process of allocating and freeing up memory to ensure efficient use of system resources during the execution of a program. JavaScript uses an automatic garbage collection mechanism to handle memory management, which helps in cleaning up memory that is no longer in use. The garbage collector automatically identifies and removes objects that are no longer reachable or needed by the program, freeing up memory for future use.

Garbage collection is the process by which the JavaScript engine automatically detects and removes objects that are no longer referenced or needed in the program. This is done using techniques like reference counting or mark-and-sweep. In mark-and-sweep, for instance, the garbage collector marks objects that are still in use and then sweeps through the memory to remove those that are not marked. This ensures that memory is efficiently managed without requiring manual intervention from the programmer.

However, memory management can sometimes lead to **memory leaks**, which occur when objects are no longer needed but are still referenced, preventing the garbage collector from reclaiming that memory. This can lead to excessive memory usage, performance degradation, and even crashes in long-running applications. Memory leaks can happen due to unintentional references, such as global variables, closures that retain unnecessary references, or event listeners that are not properly removed. To prevent memory leaks, it's important to be mindful of how objects are referenced and ensure proper cleanup of unused resources.

52) **Debouncing** is a programming technique used to control how often a function is executed, particularly in scenarios where an event is triggered frequently in a short span of time. The primary goal of debouncing is to ensure that a function is called only once after a specific delay, regardless of how many times the event occurs during that period. This is especially useful in optimizing performance for tasks like resizing windows, scrolling, searching, or keypress events where executing a function repeatedly could lead to unnecessary computations or degraded user experience.

For example, consider a search bar where every keypress initiates a network request to fetch results. Without debouncing, each keypress would send a request, leading to a flood of unnecessary calls. With debouncing, the function waits for a specified delay after the last keypress before executing, ensuring only one request is sent.

53) **Throttling** is a programming technique used to limit the frequency at which a function is executed over time, ensuring it runs at regular intervals, even if an event occurs continuously. Unlike debouncing, which delays execution until an event stops firing, throttling guarantees that the function executes periodically, regardless of how often the event is triggered. This is useful in scenarios like scrolling, resizing, or mouse movement, where continuously firing a function can overwhelm the browser or system. For example, if you want a function to execute at most once every 500 milliseconds during a scroll event, throttling ensures that it adheres to this limit, improving performance and maintaining a smooth user experience.

54) A **recursive function** is a function that calls itself in order to solve a problem. It breaks down a complex problem into smaller, simpler sub-problems, which are easier to solve. The function continues calling itself until it reaches a **base case**, which stops the recursion. Recursion is often used to solve problems like calculating factorials, generating Fibonacci numbers, or traversing hierarchical structures like trees.

Example:

```
function factorial(n) {
  if (n === 0) { // Base case
    return 1;
  } else {
    return n * factorial(n - 1); // Recursive call
  }
}
console.log(factorial(5)); // Output: 120
```

In this example, the function factorial() keeps calling itself until it reaches the base case (n === 0), and then the recursive calls start returning their results.

---

55) eval() Function

The eval() function is a built-in JavaScript function that evaluates or executes a string of JavaScript code. It takes a string as an argument, parses it as JavaScript, and then executes it. While eval() can be powerful, it is also considered dangerous because it can execute arbitrary code, potentially leading to security vulnerabilities, especially when handling untrusted input.

Example:

```
let x = 10;
let code = 'x + 5';
console.log(eval(code)); // Output: 15
```

In this example, eval() evaluates the string 'x + 5' and returns 15. Despite its utility, it's recommended to avoid using eval() whenever possible, especially with untrusted data, as it can lead to security risks like code injection.

56) A **polyfill** is a piece of code (usually JavaScript) that provides functionality that is not natively supported by a browser or environment. It allows you to use modern features, such as new JavaScript methods or APIs, in older browsers that do not support them. Polyfills are used to ensure cross-browser compatibility and make sure that new features work on older versions of browsers.

57) The **nullish coalescing operator (??)** is a logical operator in JavaScript that returns the right-hand operand if the left-hand operand is null or undefined. Unlike the logical OR (||) operator, which returns the right-hand operand if the left-hand operand is falsy (i.e., false, 0, NaN, "", null, undefined), the nullish coalescing operator only checks for null or undefined.

This is useful when you want to provide a default value only when the variable is null or undefined, without affecting other falsy values like 0 or "".